

PROJECT ASSIGNMENT

HOSPITAL MANAGEMENT SYSTEM

Object-Oriented Programming | Total: 10.0 Marks

Student name:		Student ID:	
Class:		Submission date:	

1. Problem Description

An organization needs a Hospital Management System to manage patient records and basic hospital admission information. Students must analyze the problem, design an object-oriented solution, and implement a complete menu-driven application.

The system must support the following functions:

- Add a new patient.
- Delete an existing patient.
- Edit and update patient information.
- Search for patients and display information.
- Save patient data to a file.
- Load patient data from a file.

Students must select appropriate classes, attributes, methods, relationships, data structures, validation rules, and a file format. Each patient should contain sufficient information, such as Patient ID, full name, date of birth, gender, address, phone number, diagnosis, doctor, and admission status. Additional attributes may be introduced when justified.

2. Analysis and UML Design

Before implementation, analyze the requirements and submit a UML Class Diagram with a brief explanation of the main design decisions.

- Identify the main classes/entities and their responsibilities.
- Specify attributes and operations for each class.
- Show visibility, data types, relationships, and multiplicities where appropriate.
- Ensure that the implementation is consistent with the submitted diagram.
- There is no predefined class structure; any reasonable and consistent OOP design is acceptable.

Mark: 1.5

3. Functional Requirements

3.1 Add and Manage Patient Data

Allow the user to add a new patient. All required data must be entered; Patient ID must be unique; invalid values must be rejected; and a meaningful success or error message must be displayed.

- Mandatory text fields must not be empty.
- Numeric and date values must use valid ranges and formats.
- Domain-specific values and statuses must be validated.

Mark: 1.0

3.2 Delete, Edit, and Update Patient Data

- Delete a patient using its Patient ID after locating the record.
- Edit one or more attributes of an existing patient.
- Prevent invalid updates and preserve Patient ID uniqueness.
- Handle a non-existing Patient ID without terminating the program.
- Display a clear result message after each operation.

Mark: 1.5

3.3 Search and Display Information

Implement useful search criteria, including Patient ID, patient name, diagnosis, assigned doctor, and admission status. Text searches should support reasonable matching, such as case-insensitive partial matching.

- Display one selected patient.
- Display all patients in a clear and readable format.
- Show a suitable message when no patient matches or the collection is empty.

Mark: 1.5

3.4 Save Data to File

- Save all current patient records to an appropriate file.
- Preserve enough data to reconstruct each patient correctly.
- Handle file-writing errors appropriately.
- Describe the selected file format in the README.

Mark: 1.0

3.5 Load Data from File

- Read patient data and reconstruct records correctly.
- Store loaded patients in the application's collection.
- Handle missing, empty, invalid, or corrupted files.
- Avoid loading invalid records or creating duplicate Patient ID values.

Mark: 1.0

4. Data Validation and Error Handling

Validate input and handle exceptional situations, including duplicate Patient IDs, invalid dates or phone numbers, empty required medical data, and unsupported admission status values, incorrect menu input, empty collections, missing files, and malformed file data. The application must not terminate unexpectedly because of invalid input.

Mark: 0.5

5. Object-Oriented Requirements

The solution must demonstrate appropriate object-oriented analysis and implementation. Avoid placing all functionality in one class.

- Encapsulation and appropriate class responsibilities.
- Relationships between objects and abstraction where appropriate.
- Inheritance and polymorphism where they genuinely improve the design.
- Separation of concerns, reusable methods, and suitable data structures.
- Meaningful naming, maintainable code, and consistency with the UML diagram.

Mark: 2.0

6. User Interface

Provide a clear, consistent, menu-driven interface that exposes every required function and displays meaningful feedback. A possible menu is shown below; students may design an equivalent interface.

```
===== HOSPITAL MANAGEMENT SYSTEM =====  
1. Add a new patient  
2. Delete a patient  
3. Edit/Update a patient  
4. Search for patients  
5. Display a patient  
6. Display all patients  
7. Save patients to file  
8. Load patients from file  
0. Exit  
Enter your choice:
```

7. Marking Scheme

No.	Assessment Criteria	Marks
1	Problem analysis and UML design	1.5
2	Add and manage patient data	1.0
3	Delete, edit, and update patient data	1.5
4	Search and display information	1.5
5	Save data to file	1.0
6	Load data from file	1.0
7	Data validation and error handling	0.5
8	OOP design, code organization, and code quality	2.0
	TOTAL	10.0

8. Expected Deliverables

- Complete application source code.
- UML Class Diagram.
- Sample data file used for testing.
- A short README containing the system overview, class responsibilities, relationships, compilation and execution instructions, file format, and major design decisions.

9. Evaluation Focus

The project is evaluated for functional correctness and for the student's ability to analyze requirements, identify suitable objects and relationships, design UML, apply OOP principles, validate input, handle errors, persist and restore data, and write clear, reusable, maintainable code.

Final Score: 10.0 Marks